

Syllabus: UNIT –I

Concepts of OOP: Introduction OOP, Procedural vs. Object Oriented Programming, Principles of OOP, Benefits and applications of OOP. C++ Basic Overview, Program structure, namespace, identifiers, variables, constants, enum, operators, typecasting, control structures.

Introduction:

Program: A program is a set of instructions that a computer follows in order to perform a particular task. Programmers write instructions in various programming languages to perform their computation tasks.

Various types of Languages are:

- (i) Machine level Language
- (ii) Assembly level Language
- (iii) High level Language

Machine level Language :

Machine Language is a collection of binary digits (01001101) or bits . Machine Language is the only language a computer understands.

Most machine languages consist of binary codes for both data and instructions. E.g., to add two

numbers we would need a series of binary codes such as:

0010 0000 0000 0100

0100 0000 0000 0101

0011 0000 0000 0110

Assembly level Language :

An assembly language is a low-level programming language designed for a specific type of processor.

It uses mnemonic codes(MOV,ADD,JMP....) that is English-like abbreviations to represent the machine-language instructions.

Assembly language has to be converted into executable machine code , a translator program called an assembler is used to convert each instruction from the assembly language to the machine language.

for E.g.: To add two numbers A and B in assembly lang

we use LOAD instruction, ADD instruction to add

LOAD

R1, A

LOAD

R2 , B

ADD

R1, R2

High level Language :

High-level language is more English-like.

Each high-level language has to be translated to low-level language .

We Use compilers to translate high-level language into machine language. Compilers translate the whole program first, then execute the object program.

E.g., To add two numbers A and B simply we can use '+' operator
read two values in to A and B
 $\text{Sum} = A + B$

A programming language such as C, enables to write programs which is understandable to programmer(human) and can perform any sort of task. such languages are considered as High-level because they are close to human languages. In contrast, assembly languages are considered low-level because they are very close to machine languages.

The first high-level programming languages were designed in the 1950s.

Examples : Ada , Algol, BASIC, COBOL, C, C++, JAVA, Python, FORTRAN, LISP, Pascal, Prolog.

The high-level programming languages are broadly categorized in to two categories:

- (i) Procedure oriented programming (POP) language.
- (ii) Object oriented programming (OOP) language.

Procedure Oriented Programming Language

In the procedure oriented approach, the problem is viewed as sequence of things to be done such as reading , calculating and printing. A number of functions are written to accomplish this task.

Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and organizing these instruction into groups known as functions.

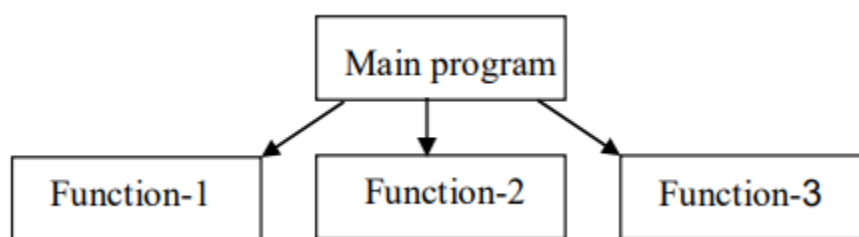


fig : structure of procedure oriented programming

The disadvantages of the procedure oriented programming languages is:

1. Global data access.
2. It does not model real word problem very well.
3. No data hiding.
4. Large size programs are divided in to smaller programs known as functions these functions can call one another . Hence security is not provided.

5. Another main problem was data security. Importance is not given to protection of data.
6. Data passes globally from one function to another function.

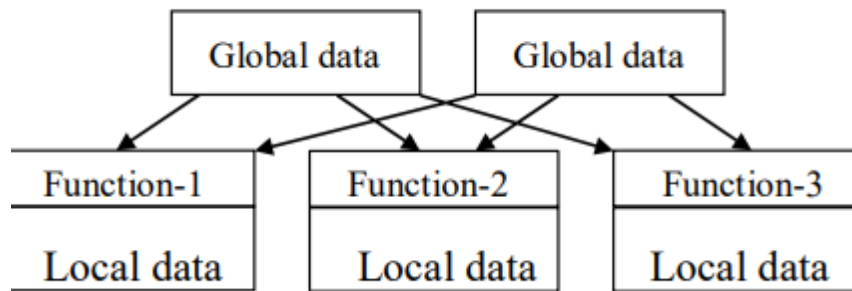


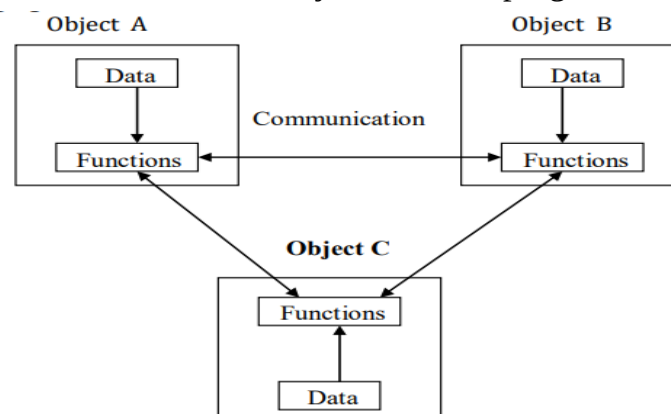
fig: Relationship of data and functions in procedure programming

Characteristics of procedure oriented programming:

1. Emphasis is on doing things (algorithm).
2. Large programs are divided into smaller programs known as functions.
3. Most of the functions share global data.
4. Data move openly around the system from function to function.
5. Function transforms data from one form to another.
6. Employs top-down approach in program design

Object Oriented Programming / Object-oriented technology (OOT)

- The major motivating factor in the invention of object oriented approach is to overcome the draw backs of procedural oriented approach.
- The principal of Object Oriented Programming Approach is to combine both data and functions into a single unit. such a single unit is called object.
- Object oriented programming treats data as critical element in the program development and does not allow it freely to flow around the system or application or program.
- Object oriented programming combines both data and functions in to a single unit this process is called Encapsulation.
- Ex : languages like C++, JAVA, PYTHON follow this approach.
- The organization of data and functions in Object oriented programming is shown in the following figure.



Features of the Object Oriented programming

1. In OOP importance is given to data rather than procedure.
2. Programs are divided into what are known as objects.
3. New data and functions can be easily added whenever necessary.
4. Data of an object can be accessed only by functions associated with that object.
5. Data is hidden and can't be accessed by external functions.
6. Objects may communicate with each other through functions.
7. Follows bottom-up approach in program design.

Structure of C Program:

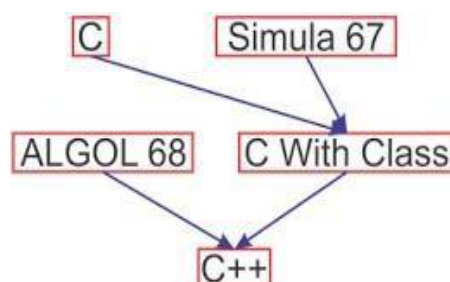
Collection of functions
Void main() ← Entry point
{
call the functions from here
}

Structure of C++ Program:

class Example
{
Collection of functions and variables
};
Void main() ← Entry point
{
create the object of the class
using the object invoke the functions in the class
}

Evolution of C++:

- C++ is an object oriented programming language and is considered as an extension of C.
- C++ was developed by Bjarne Stroustrup in 1979, at AT & T Bell labs.
- Initially stroustrup combined the features of C programming language as well as Simula67(a first object oriented language) for developing a most powerful Object oriented programming language it was called "C with classes" as it had all the properties of the C language.
- In 1982, Stroustrup renamed it as "C++" (++ being the [increment operator in C](#)).
- C++ introduced the concept of Class and Objects.
- It encapsulates high and low-level language features. So, it is seen as an intermediate level language.



C program using function:

```
#include<stdio.h>

void printName();

void main ()
{
    printf("Hello ");           //OP : Hello
    printName();                C PROGRAM
}

void printName()
{
    printf("C PROGRAM");
}
```

C++ program using function:

```
#include <iostream>
using namespace std;
// declaring a function
void greet() {
    cout << "Hello there!";
}

int main() {
    // calling the function
    greet();
    return 0;
}
```

The following are the differences between C and C++

No.	C Procedure Language	C++ Object Oriented Language
1)	C follows the procedural style programming .	C++ is multi-paradigm. It supports both procedural and object oriented .
2)	Data is less secured in C.	In C++, you can use modifiers for class members to make it inaccessible for outside users.

3)	C follows the top-down approach .	C++ follows the bottom-up approach .
4)	C does not support function overloading.	C++ supports function overloading. Sum(a,b), sum(a,b,c)
5)	In C, you can't use functions in structure.	In C++, you can use functions in structure.
6)	C does not support reference variables instead of this it used pointer.	C++ supports reference variables. Int x=10, int &ref =x;, ref =20;
7)	In C, scanf() and printf() are mainly used for input/output.	C++ mainly uses stream cin and cout to perform input and output operations.
8)	Operator overloading is not possible in C.	Operator overloading is possible in C++. A+b, obj1+obj2
9)	C programs are divided into procedures and modules .	C++ programs are divided into functions and classes .
10)	C does not provide the feature of namespace.	C++ supports the feature of namespace.
11)	Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.
12)	C does not support the inheritance.	C++ supports inheritance.
13)	C does not support access modifier.	C++ support access modifier. (private , public , protected)
	Example of Procedure Oriented Programming are : C , VB , FORTRAN, PASCAL , ALGOL , COBOL , BASIC	Programming are : C , VB , FORTRAN, PASCAL , ALGOL , COBOL , BASIC Example of Object Oriented Programming are : C++, JAVA, VB.NET, C#.NET , PYTHON

Key Concepts or Principles of Object Oriented Programming

Key concepts of object oriented programming are :

1. Class
2. Object
3. Data Encapsulation
4. Data Abstraction
5. Inheritance
6. Polymorphism

Class : Simply class is extension of 'C' Structure

Structure : Structure is a user defined data type .

It is a collection of heterogeneous variables.

Ex : struct stu

```
{  
    int sid, sub[5] ;  
    char sname[20];  
};
```

structure members

s1, s2, s3, s4,..... s10 ← structure variables

- Here by default all the structure members are public .
- The 'C' structure allows only variables(i.e., structure members) inside the structure. Functions are not allowed. Structure data is not protected.

C++ Structure : Structure is available in C++ language.

Ex : struct stu

```
{  
    int sid, sub[3] ;  
    char sname[20];  
    members member_functions();  
};
```

structure

- In C++ structures , member functions are introduced.
- In C++ also structure data is public, it is not secured.
- To avoid this problem they have introduced a concept called "class" in C++.

Advantages of Object Oriented Programming:

Object Oriented Programming provides several benefits to both program designer and user

- **Reusability:** In OOP programs, functions and modules that are written by a user can be reused by other users without any modification.
- **Inheritance:** Through this we can eliminate redundant code and extend the use of existing classes.
- **Data Hiding:** The programmer can hide the data and functions in a class from other classes. It helps the programmer to build the secure programs.
- **Reduced complexity of a problem:** The given problem can be viewed as a collection of different objects. Each object is responsible for a specific task. The problem is solved by interfacing the objects. This technique reduces the complexity of the program design.
- **Easy to Maintain and Upgrade:** OOP makes it easy to maintain and modify existing code as new objects can be created with small differences to existing ones. Software complexity can be easily managed.

- **Message Passing:** The technique of message communication between objects makes the interface with external systems easier.
- **Modifiability:** it is easy to make minor changes in the data representation or the procedures in an OO program. Changes inside a class do not affect any other part of a program, since the only public interface that the external world has to a class is through the use of methods.

Applications Of C++:

C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.

By the help of C++ programming language, we can develop different types of secured and robust applications:

- Window application
- Client-Server application
- Commercial applications(shopping , Banking)
- Operating systems(DOS, Windows, Linux)
- Editors(Notepad, word pad)
- Database(oracle)
- Device drivers.
- Compilers, interpreters.
- Embedded firmware (hardware supports s/w, chip s/w)etc.

C++ Program:

Before starting the abcd of C++ language, you need to learn how to write, compile and run the first C++ program.

To write the first C++ program, open the C++ console and write the following code:

```
#include <iostream.h>

#include<conio.h>

void main() {
    clrscr();
    cout << "Welcome to C++ Programming.";
    getch();
}
```

- **#include<iostream.h>** includes the **standard input output** library functions. It provides **cin** and **cout** methods for reading from input and writing to output respectively.
- **#include<conio.h>** includes the **console input output** library functions. The **getch()** function is defined in **conio.h** file.
- **void main()** The **main()** function is the entry point of every program in C++ language. The **void** keyword specifies that it returns no value.
- **cout <<"Welcome to C++ Programming."** is used to print the data **"Welcome to C++ Programming."** on the console.
- **getch()** The **getch()** function **asks for a single character**. Until you press any key, it blocks the screen.

C++ Identifiers:

C++ identifiers in a program are used to refer to the name of the variables, functions, arrays, or other user-defined data types created by the programmer. They are the basic requirement of any language. Every language has its own rules for naming the identifiers.

In short, we can say that the C++ identifiers represent the essential elements in a program which are given below:

- **Variables**
- **Constants**
- **Functions**
- **Labels**
- **Defined data types**

Some naming rules are common in both C and C++. They are as follows:

- Only alphabetic characters, digits, and underscores are allowed.
- The identifier name cannot start with a digit, i.e., the first letter should be alphabetical. After the first letter, we can use letters, digits, or underscores.
- In C++, uppercase and lowercase letters are distinct. Therefore, we can say that C++ identifiers are case-sensitive.
- A declared keyword cannot be used as a variable name.

For example, suppose we have two identifiers, named as 'FirstName()', and 'Firstname()'. Both the identifiers will be different as the letter 'N' in the first case is in uppercase while lowercase in second. Therefore, it proves that identifiers are case-sensitive

Valid Identifiers

The following are the examples of valid identifiers are:

- Result
- Test2

- `_sum`
- `power`

Invalid Identifiers

The following are the examples of invalid identifiers:

```
Sum-1 // containing special character '-'.
2data // the first letter is a digit.
break // use of a keyword.
```

The major difference between C and C++ is the limit on the length of the name of the variable. ANSI C considers only the first 32 characters in a name while ANSI C++ imposes no limit on the length of the name.

```
#include <iostream>
using namespace std;
int main()
{
    int a;
    int A;
    cout<<"Enter the values of 'a' and 'A'";
    cin>>a;
    cin>>A;
    cout<<"\nThe values that you have entered are : "<<a<<" , "<<A;
    return 0;
}
```

C++ Variable

- **Variables** are the fundamental building blocks of data manipulation and storage in programming, acting as **dynamic containers** for data in the C++ programming language.
- A **variable** is more than just a memory label. It serves as a link between abstract ideas and concrete data storage, allowing programmers to cleverly manipulate data.
- With the help of C++ variables, developers may complete a wide range of jobs, from simple **arithmetic operations** to complex algorithmic designs.
- These programmable containers can take on a variety of shapes, such as **integers**, **floating-point numbers**, **characters**, and **user-defined structures**, each of which has a distinctive impact on the operation of the program.

Programmers follow a set of guidelines when generating variables, creating names that combine alphanumeric letters and underscores while avoiding reserved keywords. More than just placeholders, variables are what drive **user input**, **intermediary calculations**, and the dynamic interactions that shape the program environment.

- A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

The example of declaring variable is given below:

```
int x;  
float y;  
char z;
```

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below

```
int x=5,b=10; //declaring 2 variable of integer type  
float f=30.8;  
char c='A';
```

Rules for defining variables.

- A variable can have alphabets, digits and underscore.
- A variable name can start with alphabet and underscore only. It can't start with digit.
- No white space is allowed within variable name.
- A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

```
int a;  
int _ab;  
int a30;
```

invalid variable names:

```
int 4;  
int hello world;  
int double;
```

C++ Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc

Data Type	Meaning	Size (in Bytes)
int	Integer	2 or 4
float	Floating-point	4

double	Double Floating-point	8
char	Character	1
wchar_t	Wide Character	2
bool	Boolean	1 (true or false)
void	Empty	0 (null value)

Types	Data Types
Basic Data Type	Int 4, char 1, float 4, double 8, long double 10 etc., long 4
Derived Data Type	array, pointer, function, reference etc.
Enumeration Data Type	Enum
User Defined Data Type	Structure, class, union

C++ Keywords: Total 95 keywords in which 30 are not available in c.

C++ keywords play a crucial role in defining the syntax and functioning of the language. They include reserved words with functions, such as specifying **data types**, managing **program flow**, and activating additional features. Understanding these terms is essential for good C++ programming and enables programmers to build **reliable** and **adaptable software**.

A keyword is a reserved word. You cannot use it as a variable name, constant name etc. **A list of 32 Keywords in C++ Language which are also available in C language are given below.**

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language which are not available in C language are given below.

_asm	dynamic_cast	namespace	reinterpret_cast	bool
------	--------------	-----------	------------------	------

explicit	new	static_cast	false	catch
operator	template	friend	private	class
this	inline	public	throw	const_cast
delete	mutable	protected	true	try
typeid	typename	using	virtual	wchar_t

Constants in C++

Constants in C++ refer to variables with fixed values that cannot be changed. Once they are defined in the program, they remain constant throughout the execution of the program. They are generally stored as read-only tokens in the code segment of the memory of the program. They can be of any available data type in C++ such as int, char, string, etc.

How to Define Constants in C++?

We can define the constants in C++ using three ways:

1. Using **const** Keyword
2. Using **constexpr** Keyword
3. Using **#define** Preprocessor

1 Using **const** Keyword

We first use the **const** keyword to make sure that our variable is a constant followed by the **datatype of the constant variable** then proceed with the **name of the constant variable** and finally give the appropriate **value to the constant variable**.

```
#include <iostream>
using namespace std;

int main()
{
    // declaring a variable
    int var = 10;

    // declaring a constant variable
    const int cons = 24;

    cout << "Initial Value:" << endl;
    cout << "var: " << var << endl;
    cout << "cons: " << cons << endl;

    // changing the value of both the constants
    var = 24;
```

```

        cons = 0; // change the constant value

        cout << "Final Value:" << endl;
        cout << "var: " << var << endl;
        cout << "cons: " << cons << endl;

        return 0;
}

```

Output

```

./Solution.cpp: In function 'int main()':
./Solution.cpp:19:10: error: assignment of read-only variable
'cons'

```

```

        cons = 0;

```

2. Constants using constexpr Keyword

The constexpr keyword is similar to const keyword and is also used to declare constants in C++. But the major difference is that the constexpr constants are initialized at compiler time, which is why their value must be known at the compile time. On the other hand, const keyword constants can be initialized at runtime and compile time.

```

// C++ program to illustrate the use of constexpr keyword
#include <iostream>
using namespace std;

```

```

int main()
{
    // defining constant
    int constexpr hoursIn_day = 24;

    // printing value
    cout << "Total hours in a day: " << hoursIn_day;
}

```

```

o/p : Total hours in a day: 24

```

3. Constants using #define Preprocessor

We can also define constants using the #define. Constants created using the #define preprocessor are called “macro constants”. Unlike the other methods, constants defined using this method simply work as an alias for their value which is substituted during preprocessing.

It is the less preferable way to define a constant in C++ due to the lack of type safety.

```
// C++ Program to illustrate the use of #define to create a
// constant
#include <iostream>
using namespace std;
// using #define to create a macro
#define Side 5
int main()
{
    // using constant
    const double area = Side * Side;
    cout << "The area of square with side 5 is ";
    cout << area;
    return 0;
}
```

Output

The area of square with side 5 is 25

C++ Operators

Operators are symbols that perform operations on variables and values.

For example, + is an operator used for addition, while - is an operator used for subtraction.

Operators in C++ can be classified into 6 types:

- [Arithmetic Operators](#)
- [Assignment Operators](#)
- [Relational Operators](#)
- [Logical Operators](#)
- [Bitwise Operators](#)
- [Other Operators](#)

1. C++ Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations on variables and data. For example,

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Operation (Remainder after division)

Increment and Decrement Operators

C++ also provides increment and decrement operators: `++` and `--` respectively.

- `++` increases the value of the operand by **1**
- `--` decreases it by **1**

2. C++ Assignment Operators

Operator	Example	Equivalent to
=	a = b;	a = b;
+=	a += b;	a = a + b;
-=	a -= b;	a = a - b;
*=	a *= b;	a = a * b;
/=	a /= b;	a = a / b;
%=	a %= b;	a = a % b;

3. C++ Relational Operators

- A relational operator is used to check the relationship between two operands. For example,
- If the relation is true, it returns 1 whereas if the relation is false, it returns 0.
- The relation is true, it returns 1 whereas if the relation is false, it returns 0.

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us false
<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us true
<code>></code>	Greater Than	<code>3 > 5</code> gives us false
<code><</code>	Less Than	<code>3 < 5</code> gives us true
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us false
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us true

4. C++ Logical Operators

Logical operators are used to check whether an expression is **true** or **false**. If the expression is **true**, it returns **1** whereas if the expression is **false**, it returns **0**.

Operator	Example	Meaning
&&	expression1 && expression2	Logical AND. True only if all the operands are true.
 	expression1 expression2	Logical OR. True if at least one of the operands is true.
!	!expression	Logical NOT. True only if the operand is false.

In C++, logical operators are commonly used in decision making. To further understand the logical operators, let's see the following examples,

C++ Bitwise Operators

In C++, bitwise operators perform operations on integer data at the individual bit-level. These operations include testing, setting, or shifting the actual bits. For example,

Operator	Description
<code>&</code>	Bitwise AND Operator
<code> </code>	Bitwise OR Operator
<code>^</code>	Bitwise XOR Operator
<code>~</code>	Bitwise Complement Operator
<code><<</code>	Bitwise Shift Left Operator

>>	Bitwise Shift Right Operator
----	------------------------------

6. Other C++ Operators

Operator	Description	Example
sizeof	returns the size of data type	sizeof(int); // 4
?:	returns value based on the condition	string result = (5 > 0) ? "even" : "odd"; // "even"
&	represents memory address of the operand	# // address of num
. (Dot)	accesses members of struct variables or class objects	s1.marks = 92;
→	used with pointers to access the class or struct variables	ptr→marks = 92;
<<	prints the output value	cout << 5;
>>	gets the input value	cin >> num;

What is Type Casting in C++?

Type casting in C++ refers to the process of converting a value of one data type to another data type. This can be useful in situations where we need to change the type of a variable to perform a certain operation or pass it to a function that requires a different data type. There are two types of type casting in C++: implicit and explicit type casting.

For example, suppose the given data is an integer type, and we want to convert it into float type. So, we need to manually cast int data to the float type, and this type of casting is called the Type Casting in C++.

```
int num = 5;
float x;
x = float(num);
//output x = 5 // it will convert in float but decimal value does not shown.
```

```
float num = 5.25;
int x;
x = int(num);
//Output: 5
```

Type Casting is divided into two types: Implicit type conversion or Implicit Type Casting and Explicit Type Conversion or Explicit Type Casting.

Implicit Type Casting or Implicit Type Conversion:

- It is known as the automatic type casting.
- It automatically converted from one data type to another without any external intervention such as programmer or user. It means the compiler automatically converts one data type to another.
- All data type is automatically upgraded to the largest type without losing any information.
- It can only apply in a program if both variables are compatible with each other.

```
#include <iostream>
using namespace std;
int main ()
{
    short x = 200; //2 bytes
    int y;        // 4 bytes
    y = x;
    cout << " Implicit Type Casting " << endl;
    cout << " The value of x: " << x << endl;
    cout << " The value of y: " << y << endl;

    int num = 20;
    char ch = 'a', 'A';
    int res = 20 + 'a';
    cout << " Type casting char to int data type ('a' to 20): " << res << endl;

    float val = num + 'A';
    cout << " Type casting from int data to float type: " << val << endl;
    return 0;
}
```

Output:

```
Implicit Type Casting
The value of x: 200
The value of y: 200
Type casting char to int data type ('a' to 20): 117    // a = 97
Type casting from int data to float type: 85         // A =65
```

Explicit Type Casting or Explicit Type Conversion

- It is also known as the manual type casting in a program.
- It is manually cast by the programmer or user to change from one data type to another type in a program. It means a user can easily cast one data to another according to the requirement in a program.
- It does not require checking the compatibility of the variables.
- In this casting, we can upgrade or downgrade the data type of one variable to another in a program.
- It uses the cast () operator to change the type of a variable.

```

#include <iostream>
using namespace std;
int main ()
{
    // declaration of the variables
    int a, b , c;
    float res;
    a = 21;
    b = 5;
    c = a/b;
    res = a/b;

    cout << " Implicit Type Casting: " << endl;
    cout << " Result: " << c << endl; // it loses some information

    cout << " \n Explicit Type Casting: " << endl;
    // use cast () operator to convert int data to float

    res = (float) 21 / 5;
    cout << " The value of float variable (res): " << res << endl;

    return 0;
}

```

Output:

```

Implicit Type Casting:
Result: 4

Explicit Type Casting:
The value of float variable (res): 4.2

```

C++ Classes and Objects

Class in C++ is the building block that leads to Object-Oriented programming.

- It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance (object) of that class.
- A C++ class is like a blueprint for an object.
- For Example: Consider the Class of Cars.
There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc.
- So here, Car is the class, and wheels, speed limits, and mileage are their properties.
- Data members are the data variables and member functions are the functions used to manipulate these variables together, these data members and member functions define the properties and behavior of the objects in a Class.
- In the above example of class Car, the data member will be wheels, speed limit, mileage, etc, and member functions can be applying start(), stop (), and move () , and increasing speed 0, etc.

An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Defining Class and Declaring Objects

A class is defined in C++ using the keyword class followed by the name of the class.

The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.

Example

program :

```
#include<iostream.h>
#include<conio.h>

class temp
{
    //public: int a;
    public: void read()
    {
        //cout<<a; //print garbage
        a=100;
        cout<<"\n"<<a;
    }
};

int main()
{
    clrscr();
    temp obj;    //object created
obj.a=10;    //error for this

    obj.read();
    getch();

    return 0; }
```

NOTE: In c++ the private data should be accessed only by member functions of the same class.

Member functions : The functions which are declared inside the class are called as member functions.

Default access specifier in C++ is private. so here 'a' is a private member .

private members are not directly accessible from out side the class. (this is called data hiding it is achieved with private declarations.)

main() is outside the class so 'a' is not accessible with object from main

o/p : 10 with public int a
100

o/p : 100 without public int a

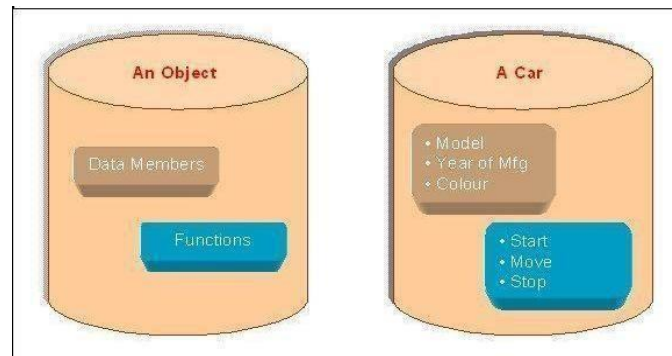
2. Object:

A class is only logical representation of data. Hence to work with the data represented by the class you must create a variable for the class which is called as an object.

So object is a variable of type class.

- Object is the basic unit of object-oriented programming.
- In a class to access data members first memory have to be allocated. No memory is allocated when a class is created.
- Memory is allocated only when an object is created, i.e., when an instance of a class is created. Object is physical copy and class is logical copy.
- Objects are identified by its unique name.
- An object represents a particular instance of a class.

There can be more than one instance of an object.
Each instance of an object can hold its own relevant data.



- An Object is a collection of data members and associated member functions also known as methods. Characteristics of an object are represented in a class as **Properties**.
- The actions that can be performed by objects becomes functions of the class and is referred to as **Methods**.
- For example consider we have a Class of *Cars* under which *Santro Xing*, *Alto* and *WaganR* represents individual Objects.
- In this context each *Car* Object will have its own, Model, Year of Manufacture, Color, Top Speed, Engine Power etc., which form **Properties** of the *Car* class and the associated actions i.e., object functions like Start, Move, Stop form the **Methods** of *Car* Class.

Encapsulation: Encapsulation in C++ is defined as the wrapping up of data and information in a single unit. In Object Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. Now,

- The finance section handles all the financial transactions and keeps records of all the data related to finance.
- Similarly, the sales section handles all the sales-related activities and keeps records of all the sales.
- **Data encapsulation:** is a mechanism of bundling the data, and the functions that use them and data abstraction is a mechanism of exposing only the interfaces and hiding the implementation details from the user.
- C++ supports the properties of encapsulation and data hiding through the creation of user-defined types, called classes. We already have studied that a class can contain private, protected and public members. By default, all items defined in a class are private. For example –

```
// C++ program to demonstrate
// Encapsulation

#include<iostream.h>
#include<conio.h>

//using namespace std;

class Encapsulation {
private:
    // Data hidden from outside world
    int x;

public:
    // Function to set value of
    // variable x
    void set(int a)
    {
        x = a;
    }

    // Function to return value of
    // variable x
    int get()
    {
        return x;
    }
};

// Driver code

int main()
{
    clrscr();
    Encapsulation obj;
    obj.set(5);
    cout<<obj.get();
    getch();
    return 0;
}
```

Output: 5

Data Abstraction: Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Consider a *real-life example of a man driving a car*. The man only knows that pressing the accelerator will increase the speed of the car or applying brakes will stop the car but he does not know how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

Types of Abstraction:

1. **Data abstraction** – This type only shows the required information about the data and hides the unnecessary data.
2. **Control Abstraction** – This type only shows the required information about the implementation and hides unnecessary information.

// C++ Program to Demonstrate the working of Abstraction

```
#include <iostream.h>
#include <conio.h>
class implementAbstraction {
private:
    int a, b;
public:
    // method to set values of
    // private members
    void set(int x, int y)
    {
        a = x;
        b = y;
    }
    void display()
    {
        cout << "a = " << a << endl;
        cout << "b = " << b << endl;
    }
};
int main()
{
    implementAbstraction obj;
    obj.set(10, 20);
    obj.display();
    return 0;
}
```

Output: a = 10 b = 20

Advantages of Data Abstraction

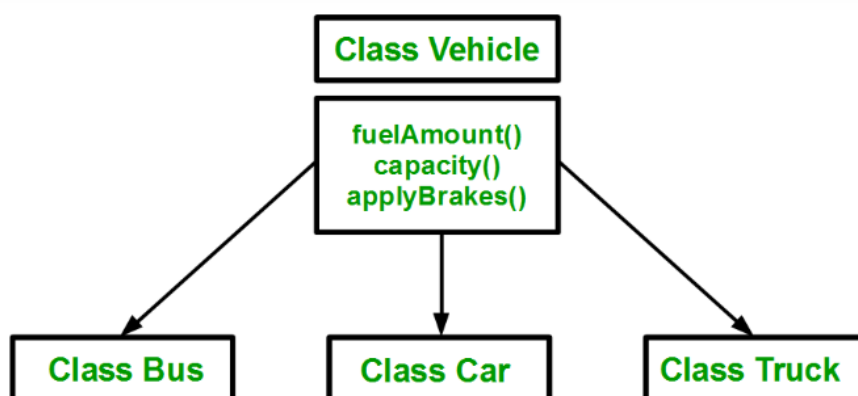
- Helps the user to avoid writing the low-level code.
- Avoids code duplication and increases reusability.
- Can change the internal implementation of the class independently without affecting the user.
- Helps to increase the security of an application or program as only important details are provided to the user.
- It reduces the complexity as well as the redundancy of the code, therefore increasing the readability.

Inheritance: The capability of a **class** to derive properties and characteristics from another class is called Inheritance.

- Inheritance is one of the most important features of Object-Oriented Programming.
- Inheritance is a feature or a process in which, new classes are created from the existing classes.
- The new class created is called “derived class” or “child class” and the existing class is known as the “base class” or “parent class”.
- The derived class now is said to be inherited from the base class.
- When we say derived class inherits the base class, it means, the derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own.
- These new features in the derived class will not affect the base class. The derived class is the specialized class for the base class.

Sub Class: The class that inherits properties from another class is called Subclass or Derived Class.

Super Class: The class whose properties are inherited by a subclass is called Base Class or Superclass.



```

// Example: define member function without argument within
// the class

#include <iostream.h>
#include <conio.h>
//using namespace std;

class person {
    int id;
    char name[100];
public:
    void set_p()
    {
        cout<< "Enter the Id:";
        cin>> id;
        cout<< "Enter the Name:";
        cin>> name;
    }
    void display_p()
    {
        cout << endl <<"Id: "<< id << "\nName: " << name <<endl;
    }   };

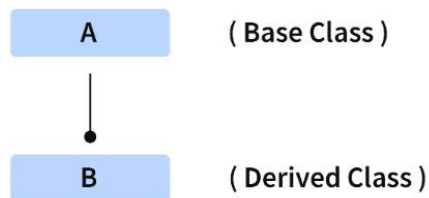
class Student : Person {
    char course[50];
    int fee;
public:
    void set_s()
    {
        set_p();
        cout << "Enter the Course Name:";
        cin >> course;
        cout << "Enter the Course Fee:";
        cin >> fee;
    }
    void display_s()
    {
        display_p();
        cout <<"Course: "<< course << "\nFee: " << fee << endl;
    }
};

int main()
{
    Student s;
    s.set_s();
    s.display_s();
    getch();
    return 0; }

```

Single Inheritance in C++:

When the derived class inherits only one base class, it is known as Single Inheritance.



```

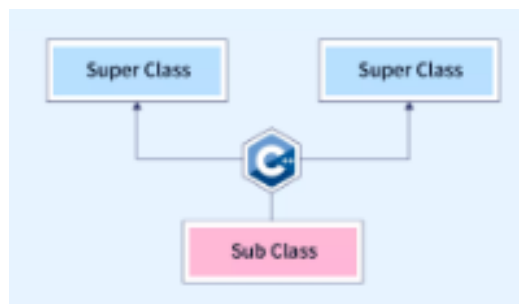
class Base {
    public:
        float salary = 900;
};
class Derived: public Base {
    public:
        float bonus = 100;
        void sum() {
            cout << "Your Total Salary is: " << (salary + bonus) << endl;
        }
};
int main() {

    // Creating an object of the derived class.
    Derived x;

    // Gets the salary variable of Base class.
    cout << "Your Salary is:" << x.salary << endl;
    // Gets the bonus variable of the Derived class.
    cout << "Your Bonus is:" << x.bonus << endl;
    x.sum();
    return 0;
}
  
```

Multiple Inheritance in C++

When a derived class (child class) inherits more than one base class (parent class), it is called multiple inheritance.



```

class Base1 {
    public:
        float salary = 900;
};

class Base2 {
    public:
  
```

```

float bonus = 100;
};

class Derived: public Base1, public Base2 {
public:
    void sum() {
        cout << "Your Total Salary is: " << (salary + bonus) << endl;
    }
};

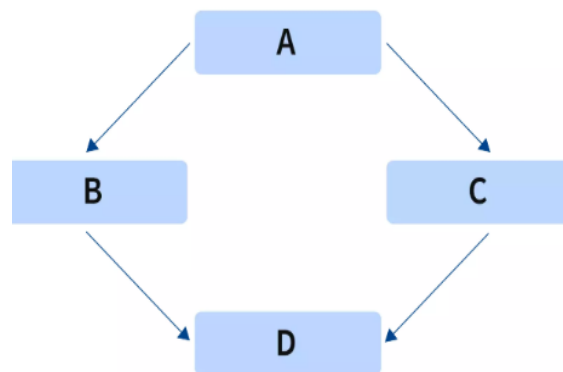
int main() {

// Creating an object of the derived class and calling the sum() function.

    Derived x;
    x.sum();
    return 0;
}

```

Diamond problem in Multiple Inheritance:



In multiple inheritance, we can face a diamond problem. Let's take an example in the above figure: "B" and "C" classes are inherited from a single Base class that is "A," and then "D" is inherited from both "B" and "C", as this situation is creating a diamond shape, therefore, this problem is known as "diamond problem".

Now here, "B" and "C" classes will have the member variable of class "A" as they are inherited from "A", as "D" is inherited from "B" and "C" so it will contain two copies of "A's" member variable that is one from "B" and one from "C", thus causing ambiguities.

The compiler will get confused about which will be taken in D from the two copies of A's member variable.

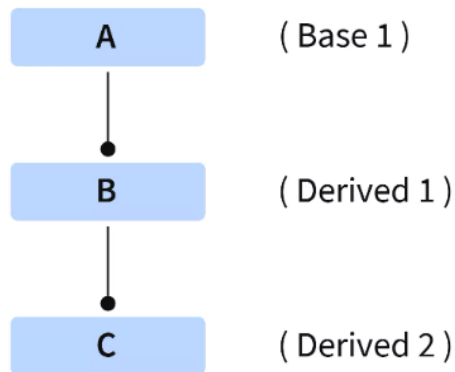
Solution of Diamond Problem:

The solution to the diamond problem is Virtual inheritance. It is a technique that ensures that only one copy of the superclasses or base class member variables is inherited by the second-level derivatives that are grandchild.

So in the above example, we can declare class "A" as a [virtual base class](#). Now only one copy of the class "A" member variable will be copied to class "B" and "C".

Multilevel Inheritance in C++:

When a derived (child) class inherits the base class and acts as the base class (parent class) to the other class, it is called Multilevel Inheritance. There can be any number of levels i.e any number of derived classes in multilevel inheritance.



In the above diagram, class "B" is derived from class "A," and class "B" is now the base class (parent class) for class "C," and class "C" is derived from class "B". Now class "C" inherits the property of Classes "A" and "B".

```
class BaseClass {
public:
    void print() {
        cout << "This is an example of Multilevel Inheritance";
    }
};

class DerivedClass: public BaseClass {};

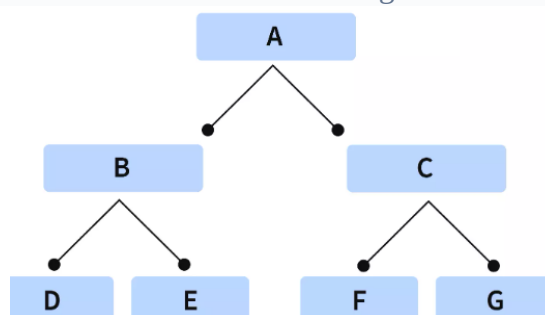
// Here class is derived from another derived class.

class DerivedClass2: public DerivedClass {};

int main() {
    DerivedClass2 Obj;
    Obj.print();
    return 0;
}
```

Hierarchical Inheritance in C++:

When more than one class is inherited from a single base class, it is called Hierarchical Inheritance.



In the above diagram, “A” is a base class, and “B” and “C” are derived classes, which inherit the parent class “A”. “D” and “E” is further derived classes that inherited the base class “B,” which is derived class to class “A”, similarly “F” and “G” are the derived class that inherits base class “C,” which is derived class to “A”.

```
class Single_base_class {
public:
    int x, y;
    void data() {
        cout << "\nEnter the value of x and y ->\n";
        cin >> x >> y;
    }
};

// Class derived1 is derived from single_base_class.
class Derived1: public Single_base_class {
public:
    void product() {
        cout << "\nThe Product is= " << x * y;
    }
};

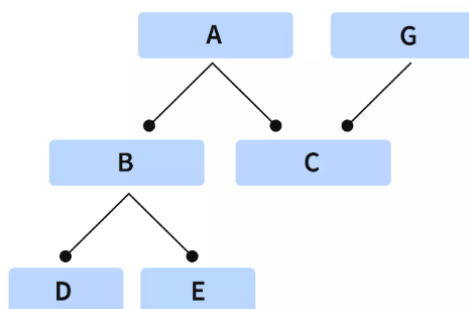
// Class derived1 is also derived from class single_base_class.
class Derived2: public Single_base_class {
public:
    void sum() {
        cout << "\nThe Sum is= " << x + y;
    }
};

int main() {

    // Creating objects of both classes and calling functions on them.
    Derived1 obj1;
    Derived2 obj2;
    obj1.data();
    obj1.product();
    obj2.data();
    obj2.sum();
    return 0;
}
```

Hybrid Inheritance in C++:

It is a combination of one or more types of inheritance.



The above diagram shows more than one type of inheritance or a combination of different types of inheritance.

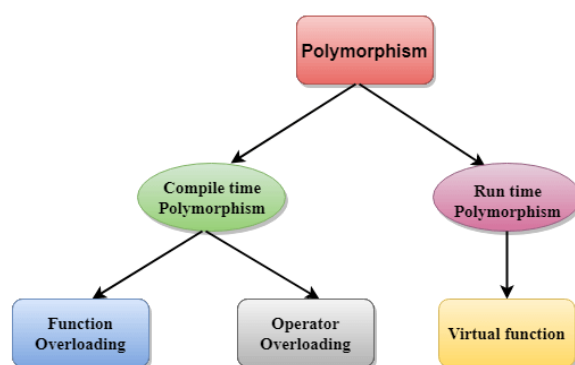
Advantages of Inheritance in C++

The advantages of inheritance are:

- Inheritance in C++ promotes Code reusability. When a derived class inherits the base class, then the derived class can access all the functionality, and the base class's code can be reused in the derived class.
- It improves code readability as you don't have to rewrite the same code repeatedly; hence, the code looks cleaner and readable.
- It saves time and effort as the code is already written and is inherited; therefore, it saves time to write code again.
- Inheritance supports extensibility as new classes can be easily added to existing classes.

6. Polymorphism :

- Polymorphism means the ability to take more than one form of any method or functions.
- Advantage of this is you can make an object behave differently in different situations, so that no need to create different objects for different situations.
- Polymorphism can be achieved with the help of Overloading and Overriding concepts and it is classified into compile time polymorphism and Runtime polymorphism.
- An example of compile time polymorphism is function overloading or operator overloading. An example of runtime polymorphism is function overriding or virtual function.



Compile time polymorphism: The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Now, let's consider the case where function name and prototype is same.

```

#include <iostream>
using namespace std;
class Example {
public :
void func(int a) {
    cout << "\nThe value of a: " << a;
}
void func(int a, int b)

{
    cout << "\nThe value of a: " << a;
    cout << "\nThe value of b: " << b;
}
void func(char c) {
    cout << "\nThe value of c: " << c;
}
};
int main() {
    Example obj;
    cout<< "\nOne int value";
    obj.func(5);
    cout<< "\nOne char value";
    obj.func('A');
    cout<< "\nTwo int values";
    obj.func(7, 2);
    return 0;
}

```

Output:

The output of the above program is as follows.

One int value

The value of a: 5

One char value

The value of c: A

Two int values

The value of a: 7

The value of b: 2

Enumeration:

- Enumerations are a user-defined data types which can be assigned some limited values.
- In this, you can specify a set of values for a variable and the variable can select only one of the set. It has a fixed set of constants.
- To define an enumeration, keyword '**enum**' is used.
- These values are defined by the programmer at the time of declaring the enumerated type.
- Enumerations can be used to define days of a week, months, weathers, etc. The enum constants are static and final implicitly.

```
enum enum_name{ element1, element2, element3, element4,};
```

```
enum season { winter, autumn, spring, summer};
```

```
enum week {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday,
Saturday};
enum month { jan to dec};

enum direction { North, South, East, West };
```

```
#include<iostream>
using namespace std;
enum week { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
int main()
{
    week day;
    day = Friday;
    cout << "Day: " << day+1<<endl;
    return 0;
}
```

Output: Day: 5

Advantages of Enumeration

Some of the advantages of enum are:

- Enum can be used in switch case.
- Enum improves type safety.
- Enum can be traversed.
- Enum can have fields, constructors and methods.
- Enum may implement many interfaces but cannot extend any class because it internally extends Enum class.

```
#include <iostream>
using namespace std;
enum direction { North, South, East, West };
int main(){
    direction dir;
    dir = East;
    //string str = to_string(dir);
    cout << "Direction: " << dir;    // op: 2
    return 0;
}
```

```
#include <iostream>
using namespace std;
int number;
int main() {
```

```
enum color {blue=1, red=2, yellow=3, green=4};
cout << "Enter one of the following numbers:" << endl;
cout << "1 2 3 4 \n";
cin >> number;
color usercolor= (color)number;
cout << usercolor; }
```

```
Enter one of the following numbers:
1 2 3 4
3

3 // yellow color chosen from set of color
```

Control Statement in C++

- A control [statement is used in a programming language](#) to control the flow of the program.
- They are nothing but a keyword or statements that are used in a program to transfer the flow of control to another statement based on the conditions.
- Based on the given condition, it evaluates the result and executes the corresponding statements.
- Control statements are the statement that controls the flow of the program in order to execute the piece of the code using various controls statement like if statement, if-else statement, break statement, continue statement, for loop, while loop, [do while loop](#).

C++ if-else

- **if statement**
- **if-else statement**
- **nested if statement**

```
#include <iostream>
using namespace std;
```

```
int main () {
    int num = 10;
    if (num % 2 == 0)
    {
        cout<<"It is even number";
    }
    return 0; }
```

Output : It is even number

C++ IF-else Statement:

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

```
#include <iostream>
using namespace std;
int main () {
    int num = 11;
    if (num % 2 == 0)
```

```

    {
        cout<<"It is even number";
    }
    else
    {
        cout<<"It is odd number";
    }
    return 0;
}

```

OUTPUT: It is odd number

C++ IF-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

```

int main() {
    double n1=20, n2=50, n3=34;

    // check if n1 is the largest number
    if (n1 >= n2 && n1 >= n3)
        cout << "Largest number: " << n1;

    // check if n2 is the largest number
    else if (n2 >= n1 && n2 >= n3)
        cout << "Largest number: " << n2;

    // if neither n1 nor n2 are the largest, n3 is the largest
    else
        cout << "Largest number: " << n3;

    getch();
    return 0;
}

```

C++ For Loop

The C++ for loop is used to iterate a part of the program several times.

```

#include <iostream>
using namespace std;
int main() {
    for(int i=1;i<=10;i++){
        cout<<i <<"\n";
    }
}

```

Output: 1 to 10

C++ Nested For Loop

In C++, we can use for loop inside another for loop, it is known as nested for loop. The inner loop is executed fully when outer loop is executed one time. So if outer loop are executed 4 times, inner loop will be executed 4 times for each outer loop i.e. total 16 times.

```
#include <iostream>
using namespace std;

int main () {
    for(int i=1;i<=3;i++){
        for(int j=1;j<=3;j++){
            cout<<i<<" "<j<<"\n";
        }
    }
}
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

C++ While loop

In C++, while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

```
#include <iostream>
using namespace std;
int main() {
    int i=1;
    while(i<=10)
    {
        cout<<i <<"\n";
        i++;
    }
}
```

Output: 1 to 10

C++ Nested While Loop Example

In C++, we can use while loop inside another while loop, it is known as nested while loop. The nested while loop is executed fully when outer loop is executed once.

Let's see a simple example of nested while loop in C++ programming language.

```
#include <iostream>
using namespace std;
int main ()
{
    int i=1;
    while(i<=3)
    {
        int j = 1;
        while (j <= 3)
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

```

    {
        cout<<i<<" "<j<<"\n";
        j++;
    }
    i++;
} }

```

C++ Do-While Loop

The C++ do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body.

```

#include <iostream>
using namespace std;
int main() {
    int i = 1;
    do{
        cout<<i<<"\n";
        i++;
    } while (i <= 10) ;
}

```

Output:

```

1
2
3
4
5
6
7
8
9
10

```

C++ Nested do-while Loop

In C++, if you use do-while loop inside another do-while loop, it is known as nested do-while loop. The nested do-while loop is executed fully for each outer do-while loop.

Let's see a simple example of nested do-while loop in C++.

```

#include <iostream>
using namespace std;
int main() {
    int i = 1;
    do{
        int j = 1;
        do{
            cout<<i<<j<<"\n";
            j++;
        } while (j <= 3) ;
        i++;
    } while (i <= 3) ;
}

```

Output:

```

1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3

```

C++ switch

The **switch statement** in C++ is a potent **control structure** that enables you to run several code segments based on the result of an expression. It offers a sophisticated and effective substitute for utilizing a succession of **if-else-if statements** when you have to make a decision between several possibilities.

The C++ switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement in C++.

```
#include <iostream>
using namespace std;
int main () {
    int num;
    cout<<"Enter a number to check grade:";
    cin>>num;
    switch (num)
    {
        case 40: cout<<"It is 40 : Grade is C "; break;
        case 50: cout<<"It is 50 : Grade is B"; break;
        case 60: cout<<"It is 60 : Grade is B "; break;
        case 70: cout<<"It is 70 : Grade is A "; break;
        default: cout<<"Error Grades are 40, 50,60 or 70"; break;
    }
}
```

Output:

```
Enter a number:
60

It is 10: Grade
is B
```

C++ Break Statement

The C++ break is used to break loop or switch statement. It breaks the current flow of the program at the given condition. In case of inner loop, it breaks only inner loop.

Let's see a simple example of C++ break statement which is used inside the loop.

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }
        cout<<i<<"\n";
    }
}
```

Output:

```
1
2
3
4
```

C++ Continue Statement

The C++ continue statement is used to continue loop. It continues the current flow of the program and skips the remaining code at specified condition. In case of inner loop, it continues only inner loop.

```
#include <iostream>
using namespace std;
int main()
{
    for(int i=1;i<=10;i++){
        if(i==5){
            continue;
        }
        cout<<i<<"\n";
    }
}
```

Output:

```
1
2
3
4
6
7
8
9
10
```

C++ Goto Statement

The C++ goto statement is also known as jump statement. It is used to transfer control to the other part of the program. It unconditionally jumps to the specified label.

It can be used to transfer control from deeply nested loop or switch case label.

```
#include <iostream>
using namespace std;
int main()
{
    NotEligible: //label also known as identifier
        cout<<"You are not eligible to vote!\n";
        cout<<"Enter your age:\n";
        int age;
        cin>>age;
        if (age < 18){
            goto NotEligible;
        }
        else
        {
            cout<<"You are eligible to vote!";
        }
}
```

Output :

```
You are not eligible to vote!
Enter your age:
16
You are not eligible to vote!

Enter your age:
7
You are not eligible to vote!

Enter your age:
22
You are eligible to vote!
```

C++ Comments

The C++ comments are statements that are not executed by the compiler. The comments in C++ programming can be used to provide explanation of the code, variable, method or class. By the help of comments, you can hide the program code also.

There are two types of comments in C++.

C++ Single Line Comment

```
int main()
{
    int x = 11; // x is a variable
    cout<<x<<"\n";
}
```

C++ Multi Line Comment

The C++ multi line comment is used to comment multiple lines of code. It is surrounded by slash and asterisk (/* */). Let's see an example of multi line comment in C++.

```
int main()
{
    /* declare and
    print variable in C++. */
    int x = 35;
    cout<<x<<"\n";
}
```

UNIT-II

Functions in C++

- A function is a set of statements that takes input, does some specific computation, and produces output.
- The idea is to put some commonly or repeatedly done tasks together to make a function so that instead of writing the same code again and again for different inputs, we can call this function.
- In simple terms, a function is a block of code that runs only when it is called.



Syntax of Function

```
// C++ Program to demonstrate working of a function
```

```
#include <iostream>
```

```
using namespace std;
```

```
// Following function that takes two parameters 'x' and 'y'
```

```
// as input and returns max of two input numbers
```

```
int max (int x, int y)
```

```
{
```

```
    if (x > y)
```

```
        return x;
```

```
    else
```

```
        return y;
```

```
}
```

```
// main function that doesn't receive any parameter and returns integer
```

```
int main()
```

```
{
```

```
    int a = 10, b = 20;
```

```
// Calling above function to find max of 'a' and 'b'
```

```
    int m = max(a, b);
```

```
    cout << "max is " << m;    // output m is 20
```

```
    return 0;
```

```
}
```

Why Do We Need Functions?

- Functions help us in reducing code redundancy. If functionality is performed at multiple places in software, then rather than writing the same code, again and again, we create a function and call it everywhere. This also helps in maintenance as we have to make changes in only one place if we make changes to the functionality in future.
- Functions make code modular. Consider a big file having many lines of code. It becomes really simple to read and use the code, if the code is divided into functions.
- Functions provide abstraction. For example, we can use library functions without worrying about their internal work.

Function Prototype in C++

- A function prototype is a declaration of the function that informs the program about the number and kind of parameters, as well as the type of value the function will return.
- One incredibly helpful aspect of C++ functions is function prototyping.
- A function prototype provides information, such as the number and type of parameters and the type of return values, to explain the function interface to the compiler.
- The prototype declaration resembles a function definition exactly, with the exception that it lacks a body, or its code. At this point, you were aware of the distinction between a statement and a definition.
 - return type
 - name of the function
 - argument list

```
int add ( int a1 , int a2 );
```

```
#include<iostream>  
using namespace std;
```

```
int add(int , int ); //declaration
```

```
int add(int x, int y) // definition  
{  
    return x + y;  
}  
int main()  
{  
    int A, B, sum;
```

```

cout << "Please enter the First Number : "<<endl;
cin >> A;
cout << "Please enter the Second Number : "<<endl;
cin >> B;

sum = add(A, B);
cout << "Sum of Two Numbers " << A <<" and " << B << " = " <<
sum;
return 0;
}

```

User-Defined Function in C++

- A **user-defined function** is a type of function in C++ language that is defined by the user himself to perform some specific task.
 - User-defined functions are different from built-in functions as their working is specified by the user and no header file is required for their usage.
1. Function with no argument and no return value.
 2. Function with no argument but return value.
 3. Function with Argument but No Return Value.
 4. Function with Argument and Return Value.

```

#include<iostream>
using namespace std;

void addTwoNumbers()
{
    double x, y;
    cout<<"Enter two Numbers: ";
    cin>>x>>y;

    cout<<endl<<"Sum of "<<x<<" and "<<y<<" is
"<<x+y;
}
int main ()
{
    addTwoNumbers();
    return 0;
}

```

```

#include<iostream>
using namespace std;

double addTwoNumbers()
{
    double x, y;
    cout<<"Enter two Numbers: ";
    cin>>x>>y;
    return (x+y);
}
int main ()
{
    double result = addTwoNumbers();

    cout<<endl<<"Sum of the numbers provided by user is:
"<<result;
    return 0;
}

```

```
#include<iostream>
using namespace std;

void addTwoNumbers (double x, double y)
{
    cout<<endl<<"Sum of "<<x<<" and "<<y<<" is "<<x+y;
}

int main ()
{
    double x, y;
    cout<<"Enter two Numbers: ";
    cin>>x>>y;

    addTwoNumbers(x, y);

    return 0;
}
```

```
#include<iostream>
using namespace std;

double addTwoNumbers(double x, double y)
{
    return x+y;
}

int main ()
{
    double x, y;
    cout<<"Enter two Numbers: ";
    cin>>x>>y;

    double result = addTwoNumbers(x, y);

    cout<<endl<<"Sum of the numbers provided by user is:
"<<result;
    return 0;
}
```

Library Functions In C++

- Library functions which are also called as “built-in” functions are the functions that are already available and implemented in C++.

```
sqrt(25);
pow(5, 3);
round(15.5);
log(x);
strcmp()
strlen()
toupper('a');
tolower()
```

```
int main() {
    int x;
    cout << "Enter any numnber ";
    cin>>x;
    // print the square root of 25
    cout << sqrt(x)<<"\n";
    cout<<pow(x,3)<<"\n";
    cout<<log(x);

    return 0;
}
```

Main Function :

- The main function is a special function. Every C++ program must contain a function named main.
- It serves as the entry point for the program. The computer will start running the code from the beginning of the main function.
- When a C++ program is executed, the execution control goes directly to the main() function.
- void:** is a keyword in C++ language, void means nothing, whenever we use void as a function return type then that function nothing return. here main() function no return any value.

- In place of void we can also use **int** return type of **main()** function, at that time main() return integer type value.
- **main**: is a name of function which is predefined function in C++ library.

```
// Without Parameters
int main()
{ ... return 0; }
```

```
// With Parameters
int main(int argc, char* const argv[])
{ ... return 0; }
```

- The reason for having the parameter option for the main function is to allow input from the command line.
- When you use the main function with parameters, it saves every group of characters (separated by a space) after the program name as elements in an array named **argv**.
- Since the main function has the return type of **int**, the programmer must always have a return statement in the code. The number that is returned is used to inform the calling program what the result of the program's execution was. Returning 0 signals that there were no problems.

C++ Pointers:

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.

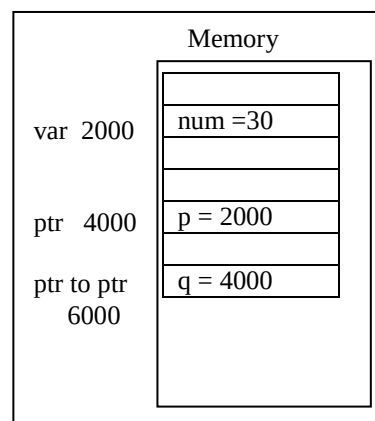
1. employing the unary operator (&), which yields the address of the variable, to assign a pointer to a variable's address.
2. Using the unary operator (*), which gives the variable's value at the address provided by its argument, one can access the value stored in an address.
3. Always assign a reference variable to the pointer . pointer will not access the simple variable.

Used of pointer

1) Dynamic memory allocation : In C++ language, we can dynamically allocate memory using new() and delete() for de-allocate, functions where pointer is used.

2) Arrays, Functions and Structures

```
#include <iostream>
using namespace std;
int main()
{
    int num=30;  9752058205
```



```
int* p;
p=&num;//stores the address of number variable
int **q =&p;
```

```
/*address of num */ cout<<"Address of number variable is:"<<&num<<endl;
```

```
cout<<"1 " <<num<<endl; //value of num 30
cout<<"2 " <<&num<<endl; // address of num 2000
cout<<"3 " <<p<<endl; //address of num which hold the pointer p 2000
cout<<"4 " <<*p<<endl; // value of num which hold the pointer p 30
cout<<"5 " <<&p<<endl; //address of pointer p 4000
cout<<"6 " <<**q<<endl; // value of pointer which hold the pointer of pointer q 30
cout<<"7 " <<*q<<endl; //address of num which hold the pointer 2000
cout<<"8 " <<q<<endl; //address of pointer which hold the q 4000
cout<<"9 " <<&q<<endl; //address of the q 6000
return 0;
}
```

```
Address of number variable is:0x7ffccc8724c4
Address of p variable is:0x7ffccc8724c4
Value of p variable is:30
Value of q variable is:30
Value of q variable is:0x7ffe5efcec08
```

Note: we can not assign simple variable to pointer type variable.

C++ References

- There is another variable that C++ supports, i.e., references. It is a variable that behaves as an alias for another variable.
- Reference can be created by simply using an ampersand (&) operator. When we create a variable, then it occupies some memory location. We can create a reference of the variable; therefore, we can access the original variable by using either name of the variable or reference. Original variable share the same address with the reference variable.

```
int a=10; // 'a' is a variable.
int &b=a; // 'b' reference to a.
int &c=a; // 'c' reference to a.

#include <iostream>
using namespace std;
int main()
{
int a=70; // variable initialization
int &b=a;
int &c=a;

int &d; //error
int *ptr ; // no error
int *ptr =&b ;
cout << "Value of a is : " <<a<<endl;
```

```
o/p : Value of a is :70
      Value of b is :70
      Value of c is :70

Value of a is :0x7ffeb57d0cdc
Value of b is :0x7ffeb57d0cdc
Value of c is :0x7ffeb57d0cdc

Value of ptr is :70
Value of ptr is :0x7ffee1470608
Value of ptr is :0x7ffeb57d0cdc

Share the same memory address for
same variable.
```

```

cout << "Value of b is :" <<b<<endl;
cout << "Value of c is :" <<c<<endl;
cout << "Value of a is :" <<&a<<endl;
cout << "Value of b is :" <<&b<<endl;
cout << "Value of c is :" <<&c<<endl;
cout << "Value of ptr is :" <<*ptr<<endl;
cout << "Value of ptr is :" <<&ptr<<endl;
cout << "Value of ptr is :" <<ptr<<endl;
return 0;}

```

Call by value in C++:

In call by value, **original value is not modified.**

When a function is called, new elements are created on the stack memory to store the essential information about the functions as well as allocated memory space for parameters and the return value.

- *The call-by-value method allows you to copy the actual parameter to a formal parameter. In this case, if we change the formal parameter then the actual parameter doesn't change.*
- In other words, the value of the parameter is duplicated into the memory location designated for the function's parameter. Consequently, two memory locations now hold the same value. C++ uses call by value method by default.

Use of Call by Value

Call by value is used in certain conditions like:

- When you do not want to change the actual parameters of the function.
- When you want to make copies of the data instead of the actual data.
- When space is not an issue.
- Usually, when you do not deal with recursion or backtracking.

```

// C++ Program to implement
// Swapping using Call by function
#include <iostream>
using namespace std;

// Swap function to demonstrate
// call by value method
void swap(int x, int y)
{
    int t = x;
    x = y;
    y = t;
    cout << "After Swapping in function x: " << x << ", y: " << y << endl;
}
// Driver Code
int main()

```

```

{
    int x = 1, y = 2;
    cout << "Before Swapping: ";
    cout << "x: " << x << ", y: " << y << endl;

    swap(x, y);
    cout << "After Swapping: ";
    cout << "x: " << x << ", y: " << y << endl;
    return 0;
}

```

Output:

Before Swapping: x: 1, y: 2
After Swapping in function x: 2, y: 1

After Swapping: x: 1, y: 2

Initialization:

```

int a = 10;
int *p = &a;
// OR
int *p;
p = &a;

```

```

int a = 10;
int &p = a; // It is
correct
// but

int &p;
p = a; // It is incorrect
as we should declare and
initialize references at
single step

```

Reassignment:

```

int a = 5;
int b = 6;
int *p;
p = &a;
p = &b;

```

```

int a = 5;
int b = 6;
int &p = a;
int &p = b; // This will throw an
error of "multiple declaration is
not allowed"

// However it is valid statement,
int &q = p;

```

Memory Address:

A pointer has its own memory address and size on the stack

reference shares the same memory address with the original variable and takes up no space on the stack.

NULL value:

null /

not null

Indirection:

int **q; // It is valid.

int &&q = p; //error

Call by reference in C++

The call by reference method of passing arguments to a function copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the passed argument.

To pass the value by reference, argument reference is passed to the functions just like any other value. So accordingly you need to declare the function parameters as reference types as in the following function swap(), which exchanges the values of the two integer variables pointed to by its arguments.

```

#include <iostream>
using namespace std;

// function declaration
void swap(int &x, int &y);

void swap(int &x, int &y) {
    int temp;
    temp = x; /* save the value at address x */
    x = y;    /* put y into x */
    y = temp; /* put x into y */

    cout << "After swap, value of x :" << x << endl;
    cout << "After swap, value of y :" << y << endl;

    //return;
}

int main () {
    // local variable declaration:
    int a = 100;
    int b = 200;

    cout << "Before swap, value of a :" << a << endl;
    cout << "Before swap, value of b :" << b << endl;

    /* calling a function to swap the values using variable reference.*/
    swap(a, b);

    cout << "After swap, value of a :" << a << endl;
    cout << "After swap, value of b :" << b << endl;

    return 0;    }

```

Before swap, value of a :100
Before swap, value of b :200

After swap, value of a :200
After swap, value of b :100

After swap, value of a :200
After swap, value of b :100

Return by reference

Return by reference is very different from Call by reference. Functions behaves a very important **role** when variable or pointers are returned as reference.

```

#include <iostream>
using namespace std;

int &max(int &x, int &y) {
    if (x > y) {
        return x;
    }
    else

```

```

return y;
}
int main() {
    int a = 10;
    int b = 20;
    int &c = max(a, b); // c is a reference to the larger of a and b

    cout << "c: " << c << endl<<endl;

    c = 30; // This sets the larger of a and b to 30
    cout << "a: " << a << endl;
    cout << "b: " << b << endl;
    cout << "c: " << c << endl;

    return 0;
}

```

c: 20

a: 10
b: 30
c: 30

In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as main().

Inline function in C++:

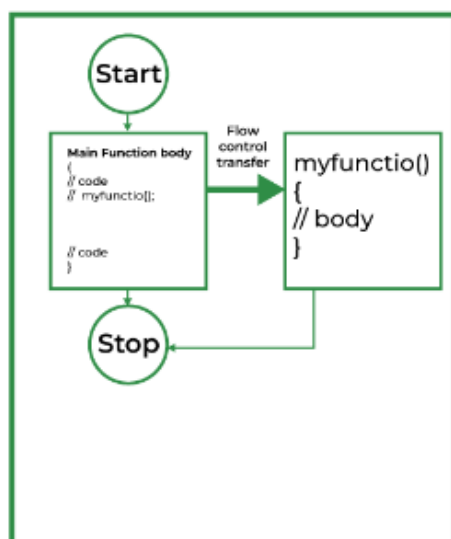
- C++ provides inline functions to reduce the function call overhead.
- An inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of the inline function call.
- This substitution is performed by the C++ compiler at compile time. An inline function may increase efficiency if it is small.

```

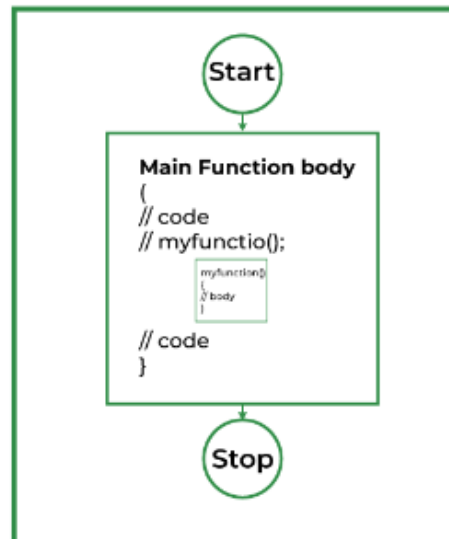
inline return-type function-name(parameters)
{
    // function code
}

```

Normal Function



Inline Function



Remember, in-lining is only a request to the compiler, not a command. The compiler can ignore the request for in-lining.

The compiler may not perform in-lining in such circumstances as:

- If a function contains a loop. (for, while and do-while)
- If a function contains static variables.
- If a function is recursive.
- If a function return type is other than void, and the return statement doesn't exist in a function body.
- If a function contains a switch or goto statement.

Inline functions Advantages:

- Function call overhead doesn't occur.
- It also saves the overhead of push/pop variables on the stack when a function is called.
- It also saves the overhead of a return call from a function.
- When you inline a function, you may enable the compiler to perform context-specific optimization on the body of the function. Such optimizations are not possible for normal function calls. Other optimizations can be obtained by considering the flows of the calling context and the called context.
- An inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function called preamble and return.

```
#include<iostream>
using namespace std;

inline int cube(int s)
{
    return s * s * s;
}

inline void printSum(int num1,int num2) {

    cout << num1 + num2 << "\n"; }

int main()
{
    cout << "The cube of 3 is: " << cube(3) << "\n";

    printSum(10, 20);

    return 0;
}
```

o/p: The cube of 3 is: 27

Inline function and classes

It is also possible to define the inline function inside the class. In fact, all the functions defined inside the class are implicitly inline. Thus, all the restrictions of inline functions are also applied here.

If you need to explicitly declare an inline function in the class then just declare the function inside the class and define it outside the class using the inline keyword.

```
class S
{
public:
    int square(int s); // declare the function
};

inline int S::square(int s) // use inline prefix
{
}
```

Inside the class : Implicitly define inline function.

```
#include<iostream>
using namespace std;

class info
{
public:

    inline int cube(int s)
    {
        return s * s * s;
    }

    inline void printSum(int num1,int num2) {
        cout << num1 + num2 << "\n"; }
};

int main()
{
    info obj;

    cout << "The cube of 3 is: " << obj.cube(3) << "\n";

    obj.printSum(10, 20);

    return 0;
}
```

**/ C++ Program to demonstrate inline functions and classes :
Explicitly define inline function.**

```
#include<iostream>

using namespace std;

class operation {
```

```

    int a, b, add, sub, mul;
    float div;

public:
    void get();           //declare all function inside the class
    void sum();
    void difference();
    void product();
    void division();
};

inline void operation::get()    //define all function outside the class
{
    cout << "Enter first value:";
    cin >> a;

    cout << "Enter second value:";
    cin >> b;
}

inline void operation ::sum()
{
    add = a + b;
    cout << "Addition of two numbers: " << a + b << "\n";
}

inline void operation ::difference()
{
    sub = a - b;
    cout << "Difference of two numbers: " << a - b << "\n";
}

inline void operation ::product()
{
    mul = a * b;
    cout << "Product of two numbers: " << a * b << "\n";
}

inline void operation ::division()
{
    div = a / b;
    cout << "Division of two numbers: " << a / b << "\n";
}

int main()
{
    cout << "Program using inline function\n";
    operation s;
    s.get();
    s.sum();
    s.difference();
    s.product();
    s.division();
    return 0;
}

```

```

Enter first value: 45
Enter second value: 15
Addition of two numbers: 60
Difference of two numbers: 30
Product of two numbers: 675
Division of two numbers: 3

```

What are Macros?

In C++, a macro is part of code that is expanded to its value. Macros are defined using the `#define` directive. They provide a way to create symbolic constants and code snippets that can be reused throughout a program.

Macros are processed by the C++ preprocessor, which runs before the actual compilation. During preprocessing, macros are replaced with their corresponding values, making it an effective tool for code generation and simplification.

Example of Macro

Let's explore a simple example of a macro in C++. Suppose we want to create a macro to calculate the square of a number. Here's how we can define and use a macro:

```
// C++ program to illustrate the macro definition
#include<iostream>
using namespace std;

// Define a macro to calculate the square of a number
#define SQUARE(x) (x * x)

int main()
{
    int n = 7;
    int result = SQUARE(n); // Expands to: (n * n)
    cout << "Square of " << n << " is " << result;
    return 0;
}
```

Square of 7 is 49

Types of Macros in C++

Macros can be classified into four types in C++:

1. Object-Like Macros `// #define PI 3.14159`
2. Function-Like Macros `// #define PRINT(x) cout << "Value is: " << x`
3. Multiline Macros `// #define PRINT_RECTANGLE(width, height)`
4. Chain Macros `// #define CLERK 10 #define WORKER CLERK`

1. Object-Like Macros

Object-like macros are simple macros that act as textual replacements. They are typically used for defining constants or short, reusable code snippets. These macros do not take arguments.

```
// C++ program to illustrate the object like macros
#include <iostream>
using namespace std;

// Define a constant for the value of PI
#define PI 3.14159

int main()
{
```

```

double radius = 4.0;

// Calculate the area of the circle
double area = PI * radius * radius;

cout << "Area of circle with radius " << radius
      << " is " << area;

return 0;
}

```

3. Function-Like Macros

Function-like macros can take arguments and perform text substitution based on those arguments. They are used for creating inline functions or code snippets, but they lack type safety.

```

// C++ program to illustrate the function like macros

#include <iostream>
using namespace std;
// Define a macro to print a value
#define PRINT(x) cout << "Value is: " << x

int main()
{
    int value = 42;
    // Print the value using the PRINT macro
    PRINT(value);
    return 0; }

```

3. Multiline Macros

Multiline macros are macros that allow us to define multiple lines of code as a single macro. They are useful for encapsulating complex code blocks, improving code readability, and reducing code duplication.

For defining multiline macros, we need to append the **(\)** **backslash** after every line.

```

// C++ program to illustrate the multiline macros
#include <iostream>

// Define a macro for printing a rectangle with a given
// width and height
Using namespace std;

#define PRINT_RECTANGLE(width, height) \

for (int i = 0; i < height; i++) { \
    for (int j = 0; j < width; j++) { \
        std::cout << "x"; \
    } \
    std::cout << std::endl; \
}

int main()
{
    // Print a rectangle with a width of 4 and a height of 3
    PRINT_RECTANGLE(4, 3);
    return 0;
}

```

4. Chain Macros

The chain macros are those macros that expand to another macro definition. For example, in a macro definition in which another macro is defined, the parent macro will be expanded first and then the child macro will be expanded.

```
// C++ program to illustrate the chain macros
#include <iostream>
using namespace std;

// defining first macro
#define CLERK 10

// defining second macro
#define WORKER CLERK

int main()
{
    // checking value of worker
    cout << WORKER;

    return 0;
}
```

What is Function Overloading in C++?

- Two or more functions can have the same name but different parameters; such functions are called function overloading in c++.
- The function overloading in the c++ feature is used to improve the readability of the code. It is used so that the programmer does not have to remember various function names.
- If any class has multiple functions with different parameters having the same name, they are said to be overloaded. If we have to perform a single operation with different numbers or types of arguments, we need to overload the function.
- If I say parameter list, it means the data type and sequence of the parameters.

For example, the parameters list of a function myfunction (int a, double b) is (int, double), which is different from the function myfunction (double a, int b) parameter list (double, int).

- Function overloading is a compile-time polymorphism. As we already know what a parameter list is, so let's see the rules of overloading: we can have the following functions in the same scope.

The easiest way to remember this rule is that the parameters should qualify any one or more than one of the following conditions:

- They should have a different type
- They should have a different number
- They should have a different sequence of parameters.

Example 1:

```
#include <iostream>
using namespace std;

void SumNum(int A, int B);
void SumNum(int A, int B, int C);
void SumNum(int A, int B, int C, int D);

int main()
{
    SumNum(1,2);
    SumNum(1,2,3);
    SumNum(1,2,3,4);

    return 0;
}

void SumNum(int A, int B)
{
    cout<< endl << "SUMNUM is : "<< A+B;
}

void SumNum(int A, int B, int C)
{
    cout<< endl << "SUMNUM is : "<< A+B+C;
}

void SumNum(int A, int B, int C, int D)
{
    cout<< endl << "SUMNUM is : "<< A+B+C+D;
}
```

SUMNUM is : 3 SUMNUM is : 6 SUMNUM is : 10
--

Example 2:

```
#include <iostream>
using namespace std;
class Addition
{
    public :
    int sum(int a, int b)
    {
        return a+b;
    }
    int sum(int a,int b, int c)
    {
        return a+b+c;
    }
};

int main()
```

```

{
    Addition obj;
    cout<<obj.sum(20, 15)<<endl;
    cout<<obj.sum(81, 100, 10);
    return 0;
}

```

error: call of overloaded 'function(double)' is ambiguous

Function Overloading and Ambiguity

- Type Conversion.
- Function with default arguments.
- Function with a pass-by reference.

1 Type Conversion.

```

#include<iostream>
using namespace std;

void function(float);
void function(int);

void function(float x)
{
    std::cout << "Value of x is : " <<x<< std::endl;
}

void function(int y)
{
    std::cout << "Value of y is : " <<y<< std::endl;
}

int main()
{
    function(3.4);
    function(34);
    return 0; }

```

error: call of overloaded 'function(double)' is ambiguous

- This example shows an error "call of overloaded 'function' is ambiguous".
- According to our prediction, function (3.4) will call the first function, and function (34) will call the second function.
- But this is not what happens because, in C++, all the floating-point constants are not treated as float; instead, they are treated as double.
- If we replace the float variable with a double variable, the program will work properly. Thus we call it a type conversion error from float to double.

Function with Default Arguments

```
#include <iostream>
using namespace std;
void fun(int);
void fun(int,int);

void fun(int i)
{
    cout << "Value of i is : " << i << std::endl;
}
void fun(int a,int b=9)
{
    cout << "Value of a is : " << a << std::endl;
    cout << "Value of b is : " << b << std::endl;
}
int main()
{
    fun(12);

    return 0;
}
```

The above example shows an error "call of overloaded 'fun(int)' is ambiguous".

The fun(int a, int b=9) can be called in two ways:

first is by calling the function with one argument, i.e., fun(12) and

another way is calling the function with two arguments, i.e., fun(4,5).

The fun(int i) function is invoked with one argument. Therefore, the compiler could not be able to select among `fun(int i)` and `fun(int a, int b=9)`.

Function with pass by reference

```
#include <iostream>
using namespace std;
void fun(int);
void fun(int &);
int main()
{
    int a=10;
    fun(a); // error, which f()?
    return 0;
}
void fun(int x)
{
    std::cout << "Value of x is : " << x << std::endl;
}
void fun(int &b)
{
}
```

```
std::cout << "Value of b is : " <<b<< std::endl;
}
```

The above example shows an error "**call of overloaded 'fun(int&)' is ambiguous**". The first function takes one integer argument and the second function takes a reference parameter as an argument. In this case, the compiler does not know which function is needed by the user as there is no syntactical difference between the fun(int) and fun(int &).

UNIT - III & IV

Access Modifiers in C++

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as **Data Hiding**.

Access Specifiers in a class decide the accessibility of the class members, like variables or methods in other classes. That is, it will decide whether the class members or methods will get directly accessed by the blocks present outside the class or not, depending on the type of Access Specifier.

In a program, we need to create methods or variables that can be accessed by the object of the same class or accessible in the entire program then Access Modifiers help us to specify that.

There are 3 types of access modifiers available in C++:

- Public
- Private
- Protected

Note: If we do not specify any access modifiers for the members inside the class, then by default the access modifier for the members will be **Private**.

1. Public: All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

```
/ C++ program to demonstrate public
// access modifier

#include<iostream>
using namespace std;

// class definition
class Circle
{
    public:
```

```

        double radius;
        double compute_area()
        {
            return 3.14*radius*radius;
        }

};

// main function
int main()
{
    Circle obj;

    // accessing public datamember outside class
    obj.radius = 5.5;

    cout << "Radius is: " << obj.radius << "\n";
    cout << "Area is: " << obj.compute_area();
    return 0;
}

```

Radius is: 5.5
Area is: 94.985

In the above program, the data member *radius* is declared as public so it could be accessed outside the class and thus was allowed access from inside main().

2. Private: The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member functions or the [friend functions](#) are allowed to access the private data members of the class.

```

#include<iostream>
using namespace std;
class Circle
{
    // private data member
    private:
        double radius;

    // public member function

public:
    double compute_area()
    {    // member function can access private
        // data member radius

        return 3.14*radius*radius;
    }    };

// main function
int main()
{
    // creating object of the class
    Circle obj;

    // trying to access private data member
    // directly outside the class
    obj.radius = 1.5;

    cout << "Area is:" << obj.compute_area();
    return 0; }

```

Error

```
In function 'int main()':  
11:16: error: 'double Circle::radius' is private  
    double radius;  
        ^  
31:9: error: within this context  
    obj.radius = 1.5;
```

The output of the above program is a compile time error because we are not allowed to access the private data members of a class directly from outside the class. Yet an access to `obj.radius` is attempted, but `radius` being a private data member, we obtained the above compilation error.

However, we can access the private data members of a class indirectly using the public member functions with argument of the class.

```
#include<iostream>  
using namespace std;  
  
class Circle  
{  
    // private data member  
    private:  
        double radius;  
  
    // public member function  
    public:  
        void compute_area(double r)  
        {    // member function can access private  
            // data member radius  
            radius = r;  
  
            double area = 3.14*radius*radius;  
  
            cout<< "Radius is: " << radius << endl;  
            cout<< "Area is: " << area;  
        }  
};  
  
// main function  
int main()  
{  
    // creating object of the class  
    Circle obj;  
  
    // trying to access private data member  
    // directly outside the class  
    obj.compute_area(1.5);  
  
    return 0;  
}
```

```
Radius is: 1.5  
Area is: 7.065
```

3. Protected: The protected access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class. The difference is that the class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

```
// C++ program to demonstrate
// protected access modifier
#include<bits/stdc++.h>

using namespace std;

// base class
class Parent
{
    // protected data members

protected:

    int id_protected;
};

// sub class or derived class from public base class
class Child : public Parent
{
public:
    void setId(int id)
    {
        // Child class is able to access the inherited
        // protected data members of base class

        id_protected = id;
    }
    void displayId()
    {
        cout<<"id_protected is: " << id_protected <<endl;
    }
};

// main function

int main() {
    Child obj1;
    // member function of the derived class can
    // access the protected data members of the base class

    obj1.setId(81);
    obj1.displayId();
    return 0;
}
```

O/P id_protected is: 81

Constructor in C++ is a special method that is invoked automatically at the time of object creation. It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class or structure. It constructs the values i.e. provides data for the object which is why it is known as a constructor.

- Constructor is a member function of a class, whose name is the same as the class name.

- Constructor is a special type of member function that is used to initialize the data members for an object of a class automatically when an object of the same class is created.
- Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object that is why it is known as a constructor.
- Constructors do not return value, hence they do not have a return type.
- A constructor gets called automatically when we create the object of the class.
- Constructors can be overloaded.
- A constructor cannot be declared virtual.

Constructors and Destructors :

Constructors and Destructors are special class operations. Constructor :

If we want to automatically initialize data members i.e., without calling member function we

want to initialize data members. It is possible only through Constructors . A constructor is a special member function.

Constructors are used to initialize the objects of its class and this is called automatic initialization of object.

Constructors constructs the values of data members of class i.e., when constructor is used data members are automatically initialized, when the object of the class is created.

Characteristics of constructors :

1. Constructor name and class name should be same.
2. Constructors should be declared in public section.
3. Constructors never have any return value (or) data type including void.
4. The main function of Constructor is to initialize objects and allocate appropriate memory to objects.
5. Constructors can have default values and can be overloaded.
6. Constructor may or may not have arguments (or) parameters.
7. Constructor is executed only once when the object is created.
8. Constructors are automatically invoked they are not invoked with dot(.) operator.
9. Constructor is invoked automatically when object of class is created.

```
// defining the constructor within the class
#include<iostream>
using namespace std;
class student {
    int rno;
    char name[50];
    double fee;

public:
    // constructor
    student()
    {
        cout << "Enter the RollNo:";
        cin >> rno;
    }
}
```

```

        cout << "Enter the Name:";
        cin >> name;
        cout << "Enter the Fee:";
        cin >> fee;
    }
    void display()
    {
        cout<<endl<<rno<<endl<<name<<endl<<fee;
    }
};
int main()
{
    student s; //constructor gets called automatically when
               // we create the object of the class
    s.display();
    return 0;
}

```

```

Enter the RollNo:121
Enter the Name:Jhon
Enter the Fee:5000
121    Jhon    5000

```

Types of Constructor in C++

Constructors can be classified based on in which situations they are being used. There are 4 types of constructors in C++:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor
4. Move Constructor

1. Default Constructor in C++

A [default constructor](#) is a constructor that doesn't take any argument. It has no parameters. It is also called a zero-argument constructor.

```

#include <iostream>
using namespace std;

class construct {
public:
    int a, b;
    construct() // Default Constructor
    {
        a = 10;
        b = 20;
    }
};
int main()
{
    // Default constructor called automatically
    // when the object is created
    construct c;
    cout << "a: " << c.a << endl << "b: " << c.b;
    return 0;
}

```

```

a: 10
b: 20

```

2. Parameterized Constructor in C++

Parameterized Constructors make it possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

Example 1: Defining Parameterized Constructor Inside the Class.

```
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }

    int getX() { return x; }
    int getY() { return y; }
};

int main()
{
    // Constructor called
    Point p1(10, 15);

    // Access values assigned by constructor
    cout << "p1.x = " << p1.getX()
         << ", p1.y = " << p1.getY();
    return 0;
}
```

p1.x = 10, p1.y = 15

A copy constructor is a member function that initializes an object using another object of the same class.

3. Copy constructor:

The process of copying one object data in to another object is called as Copy Constructor.

A copy constructor is a constructor which is used to initialize the current values with another object value.

- Copy constructor is used to create another copy or xerox of one object.
- Copy constructors are having reference type parameters.
- Copy constructors are having class type parameters means object.
- Copy constructors receives another object to initialize current object.

```

#include<iostream>
using namespace std;
class sample {

int a,b;

public:
sample(int x, int y)  // parameterized constructor
{
a=x;
b=y;
}
sample(sample &ref) // copy constructor
{
a=ref.a;
b=ref.b;
}

void display()
{
cout<<"a= "<<a<<endl;

cout<<"b= "<<b<<endl;
}
};

int main()
{
    sample s1(10,20);
    sample s2(s1);
    cout<<"Parameterized constructor"<<endl;
    s1.display();
    cout<<"copy constructor"<<endl;
    s2.display();
    return 0;
}

```

Destructors :

- Whenever constructor is executed object is defined and memory is allocated.
- Now to release (or) delete this memory we use destructor.
- Destructor is also one special member function, which is used to delete the memory created by the constructor.
- Constructor constructs the memory (or) Constructor constructs the object data.
- Destructor is used to delete this memory created by constructor.
- Using destructors is a good practice because it automatically release the memory occupied by the constructor.

Rules:

1. Destructor name should be similar to class name and preceded by 'tilde' operator(~). ex: class sample {

```

    sample() //constructor
    {
        -----
        -----
    }

    ~sample() //Destructor
    {
        -----
        -----
    }
}
;

```

2. Destructor should be declared in public section.
3. Destructor doesn't have any arguments, so overloading is not possible.
4. Destructor never returns a value, it makes a implicit call new and delete operator.
5. Destructor never called with object followed by dot(.) operator.
6. Destructor never participate in inheritance.

Program to demonstrate destructor:

```

#include<iostream>
using namespace std;
class sample {
int a,b;
public:
sample()    //constructor
{
a=10;
b=20;
cout<<"\n a= "<<a<<endl;
cout<<" b= "<<b<<endl;
}
~sample()    //destructor
{
cout<<"\n a= "<<a<<endl;
cout<<" b= "<<b<<endl;
} };

```

```

int main()
{
    sample s;
    //cout<<"\n a= "<<a<<endl;
    //cout<<" b= "<<b<<endl;
    return 0; }

```

Note: Destructor is invoked when object is delete. Before going to delete data it prints the data.

FRIEND FUNCTIONS:

The private and protected members cannot be accessed from outside the class. i.e., a non member function cannot have an access to the private data of a class.

In C++ a non member function can access private members by making the function friendly to a class.

A friend function is not a member of any class, but it is able to access the private members of those where it is introduced as a friend. when a data is declared as private inside a class, then it is not accessible from outside the class. A friend function is used for accessing the non-public members of a class. A class can allow non-member functions and other classes to access its own private data, by making them friends. Thus, a friend function is an ordinary function or a member of another class.

Definition:

A friend function is a function which is declared within a class and is defined outside the class.

It does not require any scope resolution operator for defining . It can access private members of a class. It is declared by using keyword “friend”.

When a data is declared as private inside a class, then it is not accessible from outside the class. A friend function is used for accessing the non-public members of a class. A class can allow

non-member functions and other classes to access its own private data, by making them friends.

How to declare Friend Function in C++:

Friend function declaration should be inside the class using the Friend key word.

Syntax: friend ret_type func_name (arguments);

Definition of friend function is specified outside the class and function definition must not use keyword friend.

Friend function

characterstics:

- It is not in scope of class.
- It cannot be called using object of that class.
- It can be invoked like a normal function.
- It should use a dot operator for accessing members that is inside the friend function to access the object data member we have to use obj name(dot operator).
- It can be public or private.
- It has objects as arguments.
- Friend Functions are majorly used in operator overloading.

Example to understand the friend function:

```
#include<iostream>
using namespace std;

class sample {
int a,b;

public :
void get()
{ a=100; b=200;

    cout<<"a ="<<a<<endl;
    cout<<"b ="<<b<<endl;
}

friend int compute(sample s1); };

//Friend Function Declaration with keyword friend and with the object of class sample to which it is friend
passed to it

int compute(sample s1)

{
//Friend Function Definition which has access to private data
int x = (s1.a+s1.b) - 5;
return x;
}

int main()

{
    sample s ;
    s.get();
    cout<<"The result is:"<<compute(s)<<endl;
//Calling of Friend Function with object as argument.

    return 0;}
```

The output of the above program is

a =100

b =200

The result is:295

The function compute() is a non-member function of the class sample.

In order to make this function have access to the private data a and b of class sample, it is created as a friend function for the class sample. As a first step, the function compute() is declared as friend in the class sample as:

```
friend int compute (sample s1)
```

The keyword friend is placed before the function. The function definition is written as a normal function and thus, the function has access to the private data a and b of the class sample. It is declared as friend inside the class,

In function compute the private data values a and b are added, 5 is subtracted from the result, giving 295 as the result. This is returned by the function and thus the output is displayed as shown .

// Program to access private data using non-member function, by using Friend function.

```
#include<iostream>
using namespace std;

class account {
char name[20];
int acno;
float bal;

public :
void read() {

cout<<"\n Enter Name : ";
cin>>name;
cout<<"\n Enter Account No : ";
cin>>acno;
cout<<"\n Enter Balance : ";
cin>>bal;

friend void showbal(account a);    //Friend function declaration
};

void showbal(account a) //Friend Function Definition which has access to private data
{
cout<<"\n Balance of A/c no."<<a.acno<<" is Rs." <<a.bal<<endl;
}
int main()
{
    account ac ;
    ac.read();
    showbal(ac); //Calling of Friend Function with object as argument
    return 0;
}
```

Explanation : In the above program class account is declared. It has three member variables name, acno and bal and one member function read() .

Also, inside the class account , showbal() is a function, which is declared as friend of the class account.

Once the outside function is declared as friend to any class it gets an authority to access the private data of that class. The function read() reads the data through the keyboard. The friend function showbal() display the balance and accno. inside the friend function to access the object data member we have to use obj name(dot operator)

write a program to find max of two numbers using friend function for two different classes.

```
#include<iostream>
using namespace std;

class sample1;

class sample1 {
int x;

public:
void getx(){
cout<<"Enter X : ";
cin>>x;  }

friend void max(sample1 s1,sample2 s2); };
// max is a friend function with 2 class type objects.

class sample2 {
int y;

public:
void gety() {

cout<<"Enter Y : ";
cin>>y;
}

friend void max(sample1 s1,sample2 s2);  };

void max(sample1 s1,sample2 s2) {
    if(s1.x>s2.y)
        cout<<"Data member of class sample1 is larger"<<endl;

    else if(s2.y>s1.x)
        cout<<"Data member of class sample2 is larger"<<endl;

    else cout<<"both are equal"<<endl;
}
```

```

int main()
{
    sample1 obj1;
    sample2 obj2;
    obj1.getx();
    obj2.gety();
    max(obj1,obj2);
    return 0;
}

```

Virtual Function in C++:

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call.
- They are mainly used to achieve [Runtime polymorphism](#).(function overriding)/ late binding
/ compile time poly (function overloading)/ early binding
- Functions are declared with a **virtual** keyword in a base class.

Rules for Virtual Functions

The rules for the virtual functions in C++ are as follows:

- Virtual functions cannot be static.
- A virtual function can be a friend function of another class.
- Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.
- The prototype of virtual functions should be the same in the base as well as the derived class.
- They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.
- A class may have a [virtual destructor](#) but it cannot have a virtual constructor.

Compile time (early binding) VS runtime (late binding) behavior of Virtual Functions

```

// C++ program to illustrate
// concept of Virtual Functions

#include <iostream>
using namespace std;

class base {
public:
    virtual void print() { cout << "print base class\n"; }

    void show() { cout << "show base class\n"; }
};

```

```

class derived : public base {
public:
    void print() { cout << "print derived class\n"; }

    void show() { cout << "show derived class\n"; }
};

int main()
{
    Base* bptr;
    derived d;
    bptr = &d;

    // Virtual function, binded at runtime
    bptr->print();

    // Non-virtual function, binded at compile time
    bptr->show();

    return 0;
}

```

```

print derived class
show base class

```

// C++ program to illustrate
// concept of Virtual Functions

```

#include <iostream>
using namespace std;

```

```

class base {
public:
    void print() { cout << "print base class\n"; }

    void show() { cout << "show base class\n"; }
};

class derived : public base {
public:
    void print() { cout << "print derived class\n"; }

    void show() { cout << "show derived class\n"; }
};

int main()
{
    base* bptr;
    derived d;
    // derived b;
    bptr = &d;

    // Virtual function, binded at runtime
    bptr->print();

    // Non-virtual function, binded at compile time
    bptr->show();
    return 0;}

```

```

O/P
print base class
show base class

```

Pointer to an object:

- A pointer is a variable that stores the memory address of another variable (or object) as its value. A pointer aims to point to a data type which may be int, character, double, etc.
- Pointers to objects aim to make a pointer that can access the object, not the variables. Pointer to object in C++ refers to accessing an object.
- There are two approaches by which you can access an object. One is directly and the other is by using a pointer to an object in C++.
- A pointer to an object in C++ is used to store the address of an object. For creating a pointer to an object in C++, we use the following syntax:

```
classname* pointer_to_object;
```

for ex.: `sample* sptr;`

For storing the address of an object into a pointer in c++, we use the following syntax:

```
Pointer_to_object=&objectname;
```

for ex.: `sptr = &obj;`

The above syntax can be used to store the address in the pointer to the object. After storing the address in the pointer to the object, the member function can be called using the pointer to the object with the help of an arrow operator.

```
#include<iostream>

using namespace std;

class student {
    char name[20];
    int id;
    int age;
    public:
    void getstudent()
    { cout<<"Enter Student Name: ";
      cin>>name;
      cout<<"Enter Student ID: ";
      cin>>id;
      cout<<"Enter Student Age: ";
      cin>>age; }
    void putstudent() {
```

```

cout<<"\n Student Name= "<<name<<endl;
cout<<" Student id= "<<id<<endl;
cout<<" Student Age= "<<age<<endl;
}  };

int main() {
class student s1;
student *ptr;

ptr=&s1;

ptr->getstudent();
ptr->putstudent();

s1.getstudent();
s1.putstudent();

(*ptr).getstudent();
(*ptr).putstudent();

return 0;
}

```

'This' Pointer

To better understand 'this' pointer, it is important to know how objects look at functions and data members of a class.

- Each object gets its own copy of the data member.
- All-access the same function definition as present in the code segment.

Meaning each object gets its own copy of data members and all objects share a single copy of member functions.

This is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- It can be used to pass current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.

Then now question is that if only one copy of each member function exists and is used by multiple objects, how are the proper data members are accessed and updated?

The compiler supplies an any suggested pointer along with the names of the functions as 'this'.

The 'this' pointer is passed as a hidden argument to all non-static member function calls and is available as a local variable within the body of all non-static functions. 'this' pointer is not available in static member functions as static member functions can be called without any object (with class name).

When do we use "this" keyword:

When local variable's name is same as member's name.

Example

```
#include<iostream>
using namespace std;
/* local variable is same as a member's name */
class Test
{
private:
    int m;
public:
    void setM (int m)
    {
        // The 'this' pointer is used to retrieve the object's m
        // hidden by the local variable 'm'
        this->m = m;
    }
    void print() { cout << "m = " << m << endl; }
};

int main()
{
    Test obj;
    int m = 20;
    obj.setM(m);
    obj.print();
    return 0;
}
```

Output:

m = 20

Exception Handling in C++

In C++, exceptions are runtime anomalies or abnormal conditions that a program encounters during its execution. The process of handling these exceptions is called exception handling. Using the exception handling mechanism, the control from one part of the program where the exception occurred can be transferred to another part of the code. So basically using exception handling in C++, we can handle the exceptions so that our program keeps running.

An exception is an unexpected problem that arises during the execution of a program our program terminates suddenly with some errors/issues. Exception occurs during the running of the program (runtime).

1. **Synchronous:** Exceptions that happen when something goes wrong because of a mistake in the input data or when the program is not equipped to handle the current type of data it's working with, such as dividing a number by zero.
2. **Asynchronous:** Exceptions that are beyond the program's control, such as disc failure, keyboard interrupts, etc.

Error handling mechanism

- Syntax : symbol or spelling mistake like structure or wrong in format
- Logical : wrong in conditions, loops

- Runtime: wrong input or unconditional input at the time of execution : so such type error there is a mechanism to handle.

C++ try and catch

C++ provides an inbuilt feature for Exception Handling. It can be done using the following specialized keywords: try, catch, and throw with each having a different purpose.

```
try {
    // Code that might throw an exception
    throw SomeExceptionType("Error message");
}
catch( ExceptionName e1 ) {
    // catch block catches the exception that is thrown from try block
}
```

1. try in C++

The try keyword represents a block of code that may throw an exception placed inside the try block. It's followed by one or more catch blocks. If an exception occurs, try block throws that exception.

2. catch in C++

The catch statement represents a block of code that is executed when a particular exception is thrown from the try block. The code to handle the exception is written inside the catch block.

3. throw in C++

An exception in C++ can be thrown using the throw keyword. When a program encounters a throw statement, then it immediately terminates the current function and starts finding a matching catch block to handle the thrown exception.

- Hit the exception
- Throw the exception
- Catch the exception
- Handle the exception

```
#include <iostream>
using namespace std;
int main()
{
    int a,b;

    cout<<"Enter the value of a and b ";
    cin>>a;
    cin>>b;

    int x = a/(a-b);

    cout << "Result " <<x<<endl;
}
return 0;
}
```

```
int main()
{
    int a,b;
    cout<<"Enter the value of a
and b ";

    cin>>a;
    cin>>b;

    int x = (a-b);

    try {
        if (x!= 0) {
            cout << "Result " <<a/(a-b)<<endl;
        }

        else
        {
            throw(x);
        }
    }
    catch (int i) {
        cout << "Division by zero" << endl;
    }
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {

    cout<<"Please enter the age"<<endl;
    int age=0;
    cin>>age;
    try {
        if (age<=0 || age>=100)
        {
            throw (age);
        }
    }
    catch (int e) {
        cout << "Access denied - You must enter a valid age ."<<e;
    }
    return 0;
}
```